bounded-local-ai-demo

Two small, reproducible demos behind one claim: a local AI assistant can be given useful context and real tools **and** be unable to read what it should not or run what it should not, because the surrounding system enforces the limits, not the prompt and not the model's good behavior.

It is the companion to the *Bounded Local AI Workflows* write-up, and it makes the paper's two boundaries concrete:

- **Read boundary** (`read-boundary/`) — the assistant reaches a private vault only through the `severino-vault-mcp` sensitivity gate. Enforced by the OS: this is *isolation*, not mediation.
- **Action boundary** (`action-boundary/`) — every command declares its blast radius on the Cordon effect ladder, and the gate decides `allow` / `confirm` / `block` per deployment posture.

Read boundary — what it proves

A throwaway container runs the MCP over a sample vault. `mcp` owns the vault and runs the server; `agent` is the unprivileged client. The checks run **as** `agent`:

Property	How it is enforced
Agent cannot read the vault directly	vault is `mcp` -owned, mode `go-rwx` ; agent gets permission denied
Agent cannot borrow `mcp` another way	sudoers lets agent run *only* the launcher, not `cat` / `ls` /a shell as `mcp`
Internal content IS reachable	only through the MCP, which releases `internal` bodies
Sensitive content is released, but flagged	the MCP returns the body with a handle-carefully advisory
Restricted content is withheld	the MCP sensitivity gate holds back the `restricted` doc

```
bin/bounded-local-ai-demo verify
```

Action boundary — what it proves

A stand-in `sample-tool` declares three commands at three rungs of the effect ladder. Cordon's gate maps each to a decision, and the decision depends on the posture: `local` (trusted single-operator,

fail open) vs `strict` (multi-tenant / remote, fail closed).

command	effect	local	strict
`status`	`read`	allow	allow
`build`	`local_write`	allow	confirm
`deploy`	`deploy`	confirm	block

```
bin/bounded-local-ai-demo gate
```

A `confirm` is shown as a prompt before the command runs; when there is no TTY to prompt on (a pipe, CI), it fails closed instead of running silently.

Requirements

- A container runtime exposing the `docker` CLI (Docker Desktop, Colima, OrbStack, …).
- Network access on first build (pulls the base images, clones the MCP and Cordon).

Both demos use fake, public sample data (the MCP's sample vault; a stand-in tool). No real vault, no credentials, and no host paths are baked into the images.

Quick start

```
bin/bounded-local-ai-demo build      # build both images
bin/bounded-local-ai-demo verify     # read boundary: isolation checks
bin/bounded-local-ai-demo gate       # action boundary: decision matrix
bin/bounded-local-ai-demo clean      # remove both images
```

Seeing it for yourself

Read boundary — the agent's only path to the vault is the gate. Try it directly, then through the MCP:

```
docker run --rm -it --hostname demo --entrypoint bash bounded-local-ai-demo-read
# then, as agent:
cat "/vault/02 Infrastructure/Service Handoff.md"   # Permission denied (no direct path)
```

```
mcp-read infra-service-handoff                    # released via the gate (sensitive + advisory)
mcp-read infra-offline-ca                         # withheld via the gate (restricted)
```

Action boundary — a high-blast-radius command must be confirmed, and fails closed without a TTY:

```
# confirm prompt before it runs (answer y/N):
docker run --rm -it --hostname demo --entrypoint cordon-gate bounded-local-ai-demo-gate sample-tool
deploy

# no TTY to prompt on -> blocked, not run:
docker run --rm -i --entrypoint cordon-gate bounded-local-ai-demo-gate sample-tool deploy < /dev/null

# strict posture blocks it outright:
docker run --rm -i -e CORDON_POLICY=strict --entrypoint cordon-gate bounded-local-ai-demo-gate sample-
tool deploy < /dev/null
```

How it works

```
read-boundary/      the OS-enforced isolation container
  Dockerfile          installs the MCP as mcp, copies the sample vault (go-rwx)
  run-mcp             the single channel: runs the MCP as mcp over stdio
  agent.sudoers       the lock: agent may run only run-mcp (no args) as mcp
  verify.py           the isolation checks, run as agent
  mcp-read            agent-side helper: read a doc through the gate (body or withheld)
action-boundary/    the Cordon effect-gate demo
  Dockerfile          clones cordon (zero-dependency harness), adds the sample tool
  sample-tool         declares read / local_write / deploy commands
  policy-matrix.mjs   prints the allow / confirm / block decision matrix
```

The throughline: “reachable only through the gate” and “no irreversible action without confirmation” are enforced by file permissions, a locked `sudo` entry, and a contract checked before execution — never by trusting the agent to behave.

CLI reference

Generated from the committed Cordon contract (`contract/bounded-local-ai-demo.json`), so it cannot drift from the tool. Each command declares its effect on the ladder.

`bounded-local-ai-demo`

effect: `read`

Build and run the bounded-local-AI demos (read + action boundary).

Commands

command	effect	summary
`build`	`local_write`	Build both demo images (read + action boundary).
`verify`	`local_write`	Read boundary: run the isolation checks in a throwaway container.
`gate`	`local_write`	Action boundary: print the Cordon effect-gate decision matrix.
`clean`	`local_write`	Remove both demo images.

Options

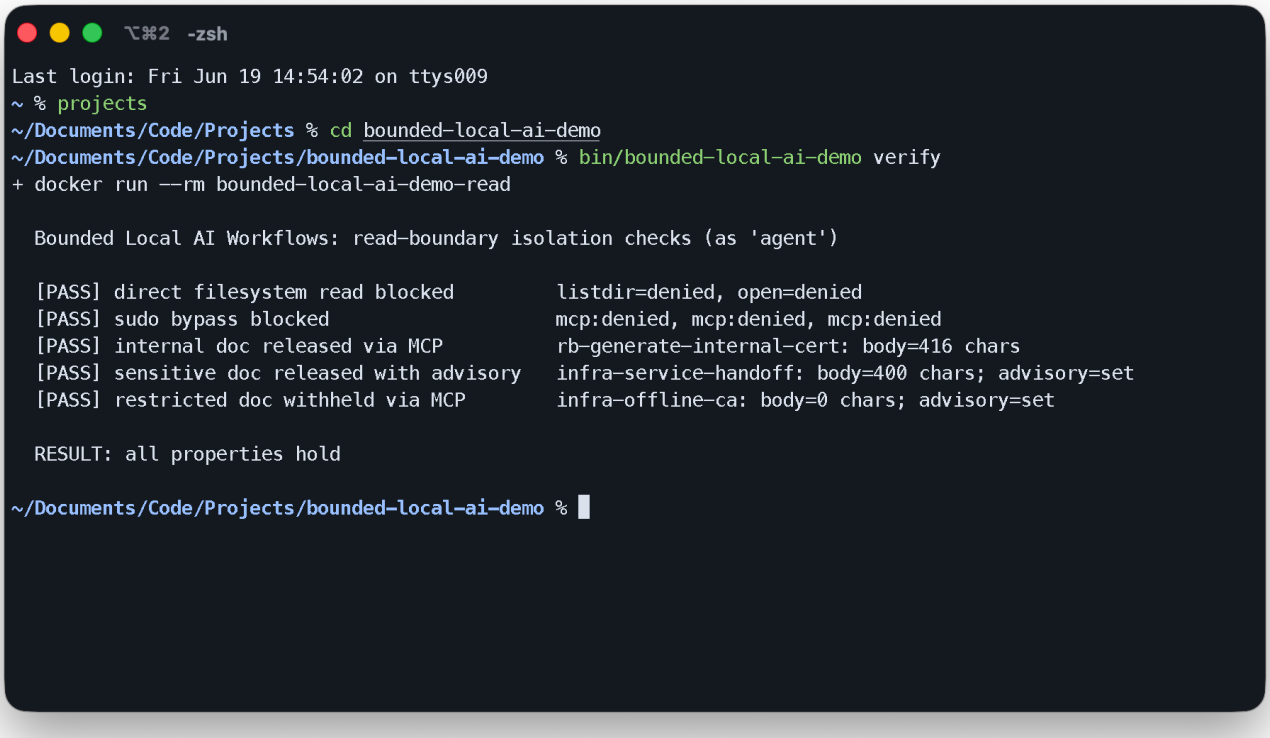
flag	value	required	help
`--describe`	no	no	Emit the Cordon v4 command-surface contract as JSON and exit.
`--pretty`	no	no	With --describe, pretty-print the JSON (default: compact).

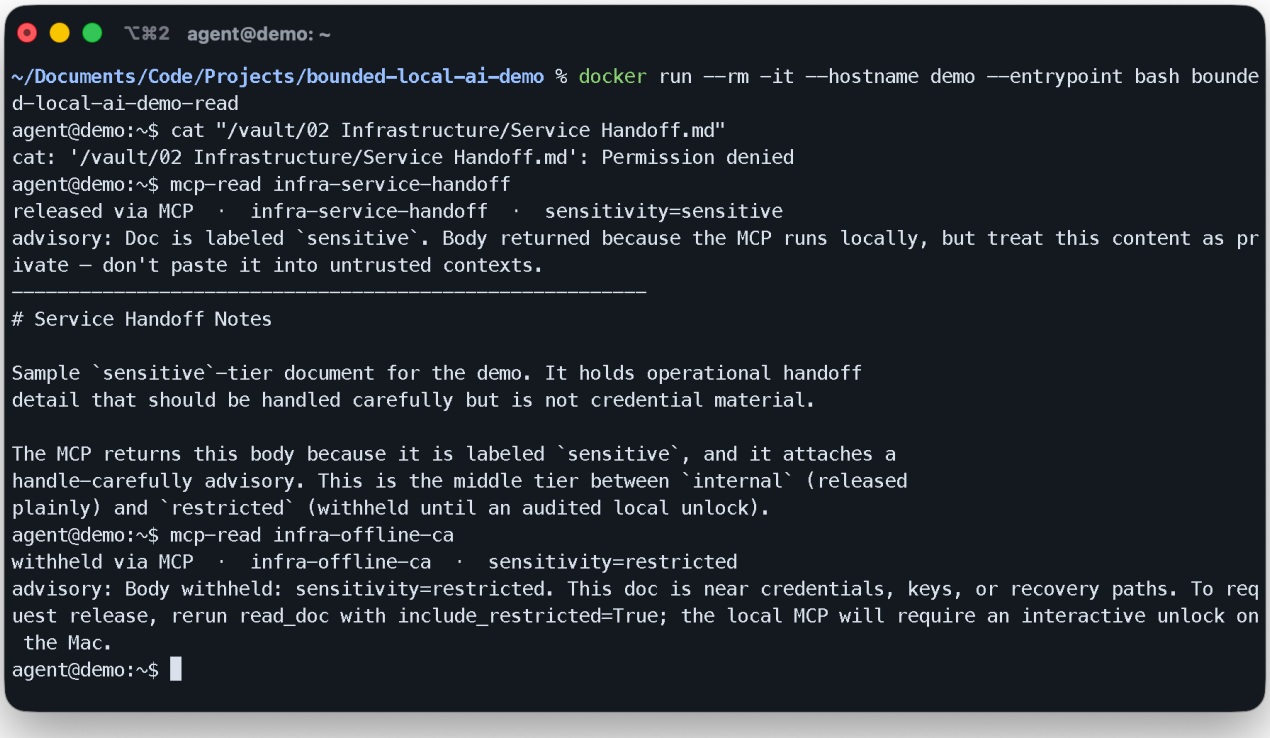
Examples

- `bounded-local-ai-demo verify` — read boundary: isolation checks
- `bounded-local-ai-demo gate` — action boundary: effect-gate matrix

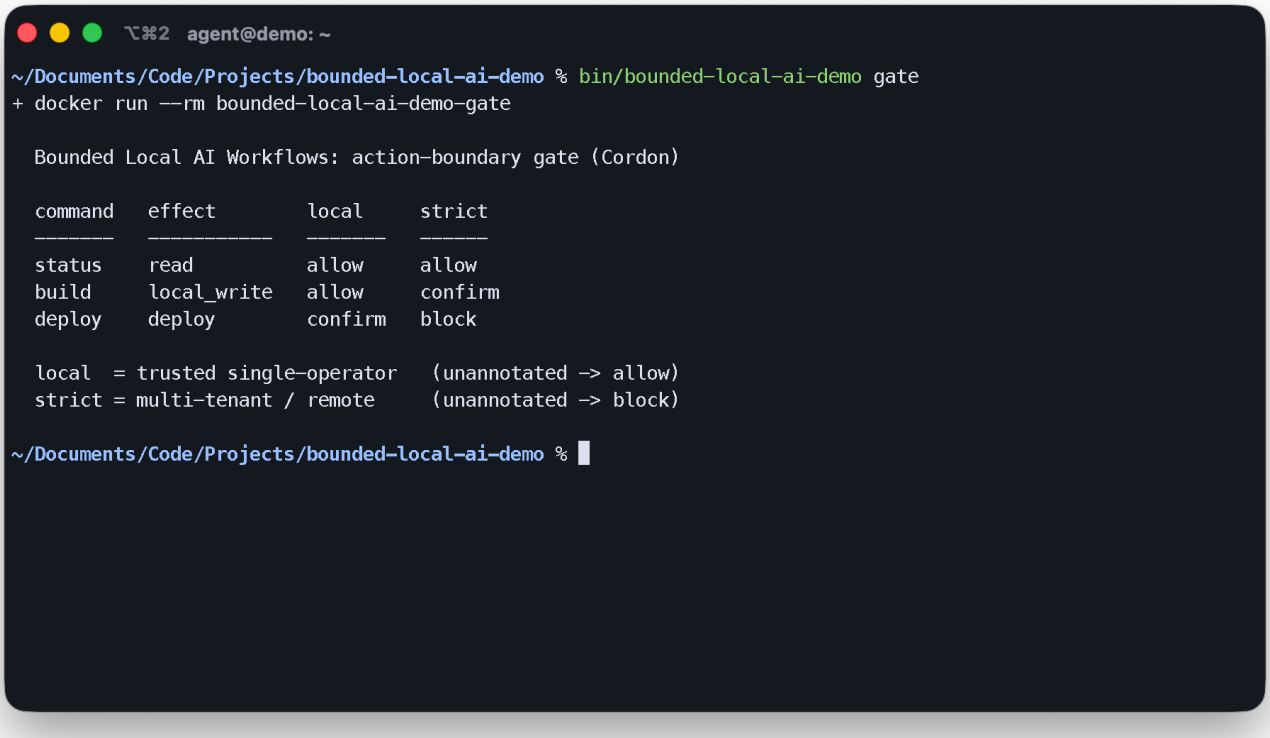
Screenshots

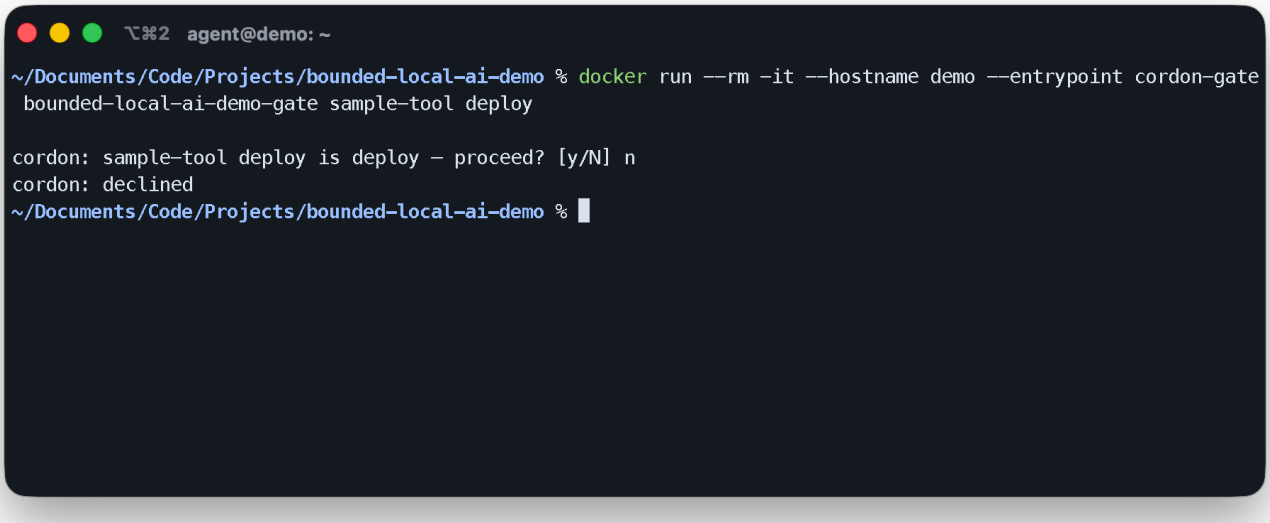
Read boundary:

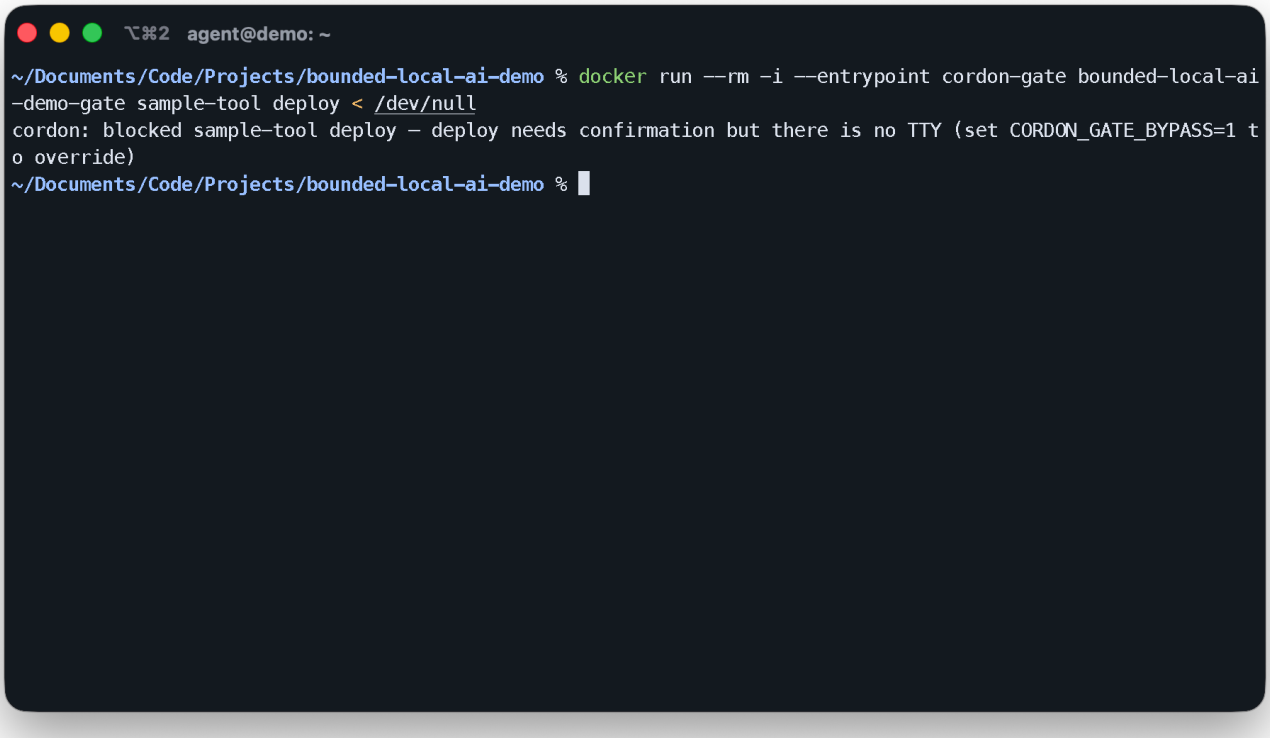




Action boundary:


⌥⌘2 agent@demo: ~
~/Documents/Code/Projects/bounded-local-ai-demo % bin/bounded-local-ai-demo gate
+ docker run --rm bounded-local-ai-demo-gate
Bounded Local AI Workflows: action-boundary gate (Cordon)
command effect local strict
status read allow allow
build local_write allow confirm
deploy deploy confirm block
local = trusted single-operator (unannotated -> allow)
strict = multi-tenant / remote (unannotated -> block)
~/Documents/Code/Projects/bounded-local-ai-demo %


⌥⌘2 agent@demo: ~
~/Documents/Code/Projects/bounded-local-ai-demo % docker run --rm -it --hostname demo --entrypoint cordon-gate
bounded-local-ai-demo-gate sample-tool deploy
cordon: sample-tool deploy is deploy — proceed? [y/N] n
cordon: declined
~/Documents/Code/Projects/bounded-local-ai-demo %



Built with

Scaffolded from cordon-starter: the Cordon command-surface contract, the green-gating CI (`cordon / gate`), release automation, and the governance setup all come from there.

License

MIT. See LICENSE.